Impariamo a ad usare eclipse e progetti maven

Ingegneria del Software AA2627

Angelo Gargantini

https://github.com/foselab/corso_unibg_ingsw_26

Obiettivo: Sviluppiamo un piccolo software – un progetto eclipse per una calcolatrice (class Calculator) che fa alcune operazioni tra numeri (somma, etc.). Nel progetto per **loggare** il comportamento del programma, inizialmente usiamo delle stampe con System.out.println, poi al suo posto useremo una libreria apposita (log4j)

1. eclipse

Scarica lo zip dal corso J: gargantini oppure da web (eclipse.org)

Usiamo il pacchetto: eclipse-java-2026-09-R-win32-x86_64.zip

Da qui Eclipse Packages | The Eclipse Foundation - home to a global community, the Eclipse IDE, Jakarta EE and over 350 open source projects...

Ricorda:

Workspace: la cartella in cui verranno salvati tutti i programmi che realizzeremo – puoi usare il desktop

Progetto: contiene tutto il codice diviso in folder (src per i sorgenti)

2. Cosa è maven?

Apache Maven è uno strumento di gestione di progetti software basati su Java e build automation.

Maven usa un costrutto conosciuto come Project Object Model (**POM**); un file XML che descrive le dipendenze fra il progetto e le varie versioni di librerie necessarie nonché le dipendenze fra di esse. In questo modo si separano le librerie dalla directory di progetto utilizzando questo file descrittivo per definirne le relazioni.

Maven effettua automaticamente il download di librerie Java e plug-in Maven dai vari repository definiti scaricandoli in locale o in un repository centralizzato lato sviluppo. Questo permette di recuperare in modo uniforme i vari file JAR e di poter spostare il progetto indipendentemente da un ambiente all'altro avendo la sicurezza di utilizzare sempre le stesse versioni delle librerie.

3. Come creare un progetto maven in eclipse

Ci sono diversi modi per creare un progetto maven in eclipse, vediamoli:

1. Creare un progetto come maven project

In questo caso scegliere come archetipo "`org.apache.maven.archetypes:maven-archetype-quickstart`"

groupId: Il Maven GroupID è l'ID dell'intero gruppo di progetto. GroupID è un'entità univoca tra tutti i progetti.

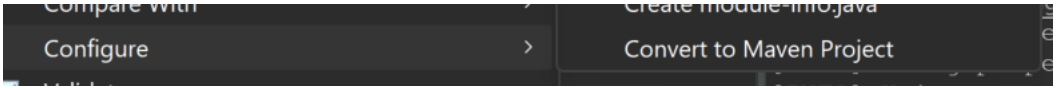
artifactId: è il nome del singolo progetto

version: 0.0.1-SNAPSHOT

Purtroppo questo schema non è aggiornato – si deve aggiornare a mano o scegliere un altro archetipo

2. Creare un Progetto eclipe e trasformarlo in maven

Si può anche creare il progetto con eclipse e poi trasformarlo in progetto maven



I due modi differiscono anche nella struttura delle directories.

ATTENZIONE: in entrambi i casi eclipse genera un file .project che contiene la descrizione del progetto per eclipse

3. In cosa è diverso un progetto maven?

Le dipendenze sono gestite (anche) da un file pom.xml

April il file pom.xml

Per editare il file pom.xml alcune volte useremo il dialogo, altre direttamente l'editor di XML:

4. Struttura delle directories di progetto maven

I progetti maven in genere hanno questa struttura delle directory. Sia **${basedir}** il folder base del progetto

Item	Default
source code	${basedir}/src/main/java
resources	${basedir}/src/main/resources
tests	${basedir}/src/test
complied byte code	${basedir}/target
distributable JAR	${basedir}/target/classes

Se avete convertito un progetto eclupse a maven la struttura sarà leggermente diversa

5. Scriviamo ed eseguiamo il codice per la calcolatrice

La classe **Calculator**, ha un costruttore che non prende argomenti e un metodo **somma** che dati due interi calcola la somma. Ha anche un **main** in cui si crea un Calculator e si fa la somma dei due numeri da te scelti, stampa il valore dei due numeri e stampa il resultato

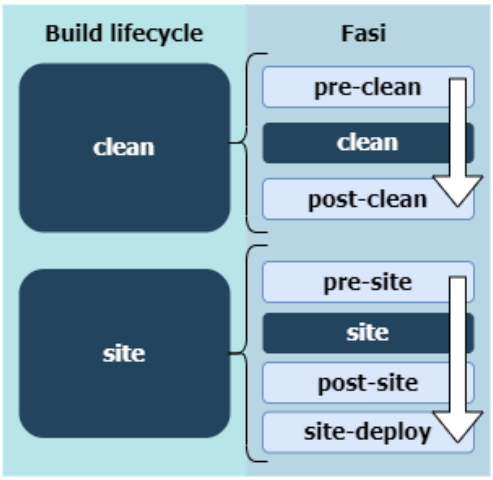
```
I due numeri sono 5 e 7
La somma è 12
```

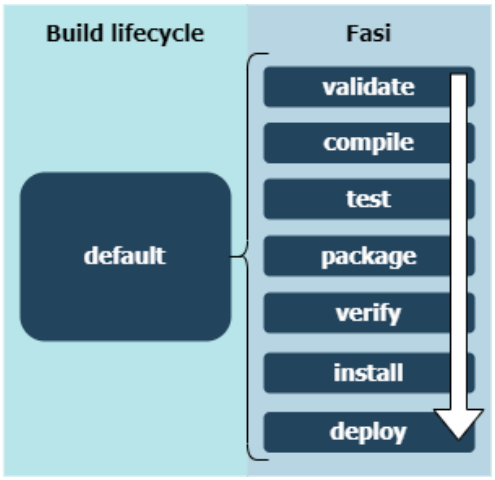
Esegui il codice (con run as)

6. Comandi maven

Maven ha già **integrati** tre **build lifecycle**:

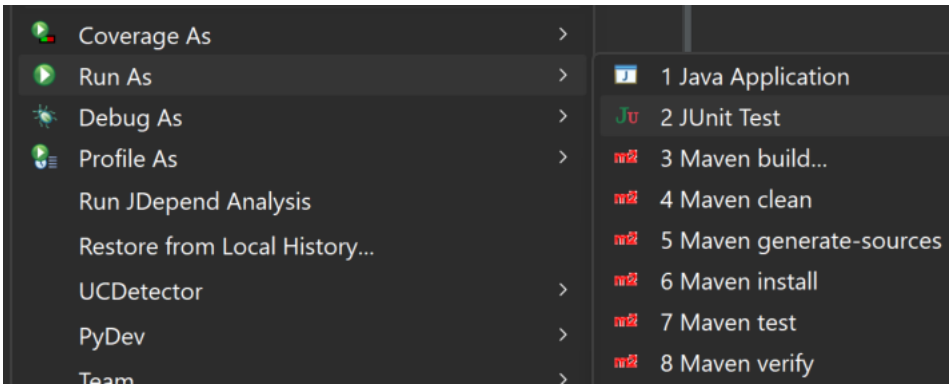
- **clean** per gestire la "pulizia" del progetto
- **site** che genera la documentazione relativa al progetto
- **default** gestisce la vera e propria "build" del progetto a partire dalla compilazione del codice fino alla creazione e distribuzione del programma eseguibile





- `validate`: verificare che il progetto sia corretto e che tutte le informazioni necessarie siano disponibili
- `compile` - compila il codice sorgente del progetto
- `test`: testare il codice sorgente compilato utilizzando un framework di unit testing adatto. Questi test non dovrebbero richiedere che il codice venga impacchettato o distribuito
- `package`: prende il codice compilato e lo impacchetta nel suo formato distribuibile, come un JAR.
- `verify`: eseguire eventuali controlli sui risultati dei test di integrazione per garantire che i criteri di qualità siano soddisfatti
- `install`: installa il pacchetto nel repository locale, per utilizzarlo come dipendenza in altri progetti locali
- `deploy`: eseguita nell'ambiente di compilazione, copia il pacchetto finale nel repository remoto per condividerlo con altri sviluppatori e progetti.

7. Come eseguire maven da eclipse



8. Usare log4j al posto di system.out

Invece di usare system.out.println usiamo una piattaforma per il logging (ce ne sono tante, usiamo log4j). Per il progetto è OBBLIGATORIO usare una piattaforma di logging.

Per istruzioni dettagliate vedi ad esempio:

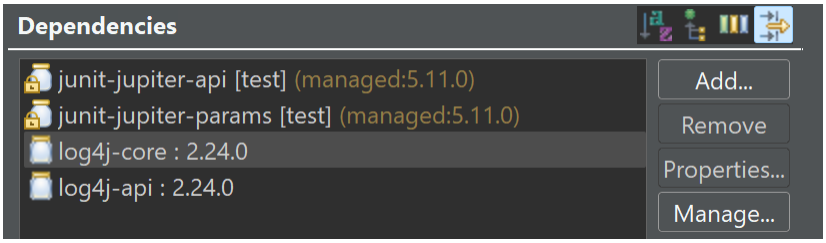
https://www.baeldung.com/java-logging-intro

9. Come aggiungere una dipendenza con maven

Aggiungiamo la libreria log4j al file pom.xml

```
<dependencies>
    <dependency>
        <groupId>org.apache.logging.log4j</groupId>
        <artifactId>log4j-api</artifactId>
        <version>2.26.1</version>
    </dependency>
    <dependency>
        <groupId>org.apache.logging.log4j</groupId>
        <artifactId>log4j-core</artifactId>
        <version>2.26.1</version>
    </dependency>
</dependencies>
```

O con il dialogo.



10. Modifichiamo il codice per avere il log.

```
import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;

/**
 * Hello world!
 *
 */
public class Calculator {

    private static final Logger LOGGER = LogManager.getLogger();

    public static void main(String... args) {
        String thing = args.length > 0 ? args[0] : "world";
        LOGGER.error("Hello, {}!", thing);
        LOGGER.debug("Got calculated value only if debug enabled: {}", () -> doSomeCalculation());
    }

    private static Object doSomeCalculation() {
        return null;
        // do some complicated calculation
    }
}
```

Nota:

per importare le classi Logger e LogManager

```
import org.apache.logging.log4j.Logger;
import org.apache.logging.log4j.LogManager;
```

per creare l'oggetto Logger:

```
private static Logger logger = LogManager.getLogger(Calcolatore.class);
```

per dare istruzioni al logger

```
logger.debug("Debug log message");
logger.info("Info log message");
logger.error("Error log message");
```

11. Eseguire

Se non c'è una configurazione specifica il logger mostra solo gli errori

12. Come configurare log4j

Vedi la documentazione della libreria.

Log4j will provide a default configuration if it cannot locate a configuration file. The default configuration, provided in the DefaultConfiguration class, will set up:

- A ConsoleAppender attached to the root logger.
- A PatternLayout set to the pattern "%d{HH:mm:ss.SSS} [%t] %-5level %logger{36} - %msg%n" attached to the ConsoleAppender

Note that by default Log4j assigns the root logger to `Level.ERROR`.